



Security Audit

Report for Tax Token

Contract

Date: October 17, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About Target Contracts	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	2
1.3.1 Security Issues	2
1.3.2 Additional Recommendation	2
1.4 Security Model	3
Chapter 2 Findings	4
2.1 Security Issue	4
2.1.1 Potential sandwich attacks due to the lack of slippage protections	4
2.1.2 Potential tax circumvention during the pool migration	5
2.1.3 Potential DoS due to the improper configuration in the contract <code>taxSplitter</code>	6
2.2 Recommendation	8
2.2.1 Correct the importing path of the contract <code>PoolAddress</code>	8
2.2.2 Remove redundant code	8
2.2.3 Emit events for the state transition	8
2.2.4 Add validity checks in the function <code>initialize()</code>	9
2.3 Note	10
2.3.1 Potential centralization risks	10
2.3.2 Proxy deployment and implementation binding should be atomic	11
2.3.3 Potential tax circumvention risks	11
2.3.4 AMM compatibility with the fee-on-transfer tokens	11
2.3.5 Proper operations during the migration process	12

Report Manifest

Item	Description
Client	Flap
Target	Tax Token Contract

Version History

Version	Date	Description
1.0	October 17, 2025	First release

Signature



Digitally signed by: BlockSec (<https://blocksec.com>)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2025-10-17 07:31:00+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The target of this audit is the code repository¹ of Tax Token Contract of Flap.

The Tax Token Contract of Flap is an ERC20 token implementing a tax mechanism. Specifically, taxes are applied to swaps in designated pools when the pool state is set to TaxEnforcedAntiFarmer and TaxEnforced. Moreover, when the accumulated tax token amount reaches a predefined threshold, a tax liquidation is triggered. In this process, the tax tokens are swapped for the quote token and transferred to the tax splitter contract. If the swap fails, the tax tokens are directly transferred to the tax splitter contract.

Note this audit only focuses on the smart contracts in the following directories/files:

- src/FlapTaxToken.sol

Other files are not within the scope of the audit. Additionally, all dependencies of the smart contracts within the audit scope are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (Version 1), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (Version 0), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
Tax Token Contract	Version 1	93642899d9e194274f21509f0f983ec70c27219e
	Version 2	c5b59dff301c03ab095617b6fd8c3048ae315036

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any war-

¹<https://github.com/flap-sh/tax-token-for-auditing.git>

ranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

- * Access control
- * Permission management
- * Whitelist and blacklist mechanisms
- * Initialization consistency
- * Improper use of the proxy system
- * Reentrancy
- * Denial of Service (DoS)
- * Untrusted external call and control flow
- * Exception handling
- * Data handling and flow
- * Events operation
- * Error-prone randomness
- * Oracle security
- * Business logic correctness
- * Semantic and functional consistency
- * Emergency mechanism
- * Economic and incentive impact

1.3.2 Additional Recommendation

- * Gas optimization

* Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology² and Common Weakness Enumeration³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we found **three** potential security issues. Besides, we have **four** recommendations and **five** notes.

- Medium Risk: 2
- Low Risk: 1
- Recommendation: 4
- Note: 5

ID	Severity	Description	Category	Status
1	Medium	Potential sandwich attacks due to the lack of slippage protections	Security Issue	Confirmed
2	Medium	Potential tax circumvention during the pool migration	Security Issue	Confirmed
3	Low	Potential DoS due to the improper configuration in the contract <code>taxSplitter</code>	Security Issue	Fixed
4	-	Correct the importing path of the contract <code>PoolAddress</code>	Recommendation	Fixed
5	-	Remove redundant code	Recommendation	Fixed
6	-	Emit events for the state transition	Recommendation	Fixed
7	-	Add validity checks in the function <code>initialize()</code>	Recommendation	Fixed
8	-	Potential centralization risks	Note	-
9	-	Proxy deployment and implementation binding should be atomic	Note	-
10	-	Potential tax circumvention risks	Note	-
11	-	AMM compatibility with the fee-on-transfer tokens	Note	-
12	-	Proper operations during the migration process	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Potential sandwich attacks due to the lack of slippage protections

Severity Medium

Status Confirmed

Introduced by `Version 1`

Description In the contract `FlapTaxToken`, the function `_liquidateTax()` is designed to liquidate the accrued tax during token transfers. When tax reaches its threshold amount or the state is updated to `PoolState.TaxFree`, all accrued tax tokens (i.e., `taxAmount`) are swapped for `QUOTE_TOKEN` via the contract `PCS_V2_ROUTER`. However, there is a lack of slippage protection for the swap, allowing attackers to gain profits by performing sandwich attacks.

```

360         try IUniswapRouter02(PCS_V2_ROUTER).
            swapExactTokensForETHSupportingFeeOnTransferTokens(
361             taxAmount, outputAmount, path, taxSplitter, block.timestamp
362         ) {

```

Listing 2.1: src/FlapTaxToken.sol

```

378         try IUniswapRouter02(PCS_V2_ROUTER).
            swapExactTokensForTokensSupportingFeeOnTransferTokens(
379             taxAmount, outputAmount, path, taxSplitter, block.timestamp
380         ) {

```

Listing 2.2: src/FlapTaxToken.sol

Impact The tax liquidation process may suffer sandwich attacks due to the lack of slippage protections.

Suggestion Add slippage protections.

Feedback from the project The project stated that they are aware of this issue and would mitigate the issue by setting reasonable values for the variables `taxRate` and `liquidationThreshold`.

2.1.2 Potential tax circumvention during the pool migration

Severity Medium

Status Confirmed

Introduced by Version 1

Description In the contract `FlapTaxToken`, the function `startMigration()` initiates pool migration by setting the variable `state` to `PoolState.Migrating`, while the function `finalizeMigration()` completes the migration by transitioning the variable `state` to `PoolState.TaxEnforcedAntiFarmer`. However, the design is improper due to the potential asynchronous execution of these state transitions. Specifically, the swaps (i.e., in the configured pools) between the start and finalization of the migration can circumvent the tax mechanism due to the pool state `PoolState.Migrating`. As a result, this improper design enables potential tax circumvention during the migration.

```

172  /// @notice Starts the migration process by transitioning the state of the pool
173  function startMigration() external override onlyOwner {
174      if (state == PoolState.BondingCurve) {
175          state = PoolState.Migrating;
176      }
177  }
178
179  /// @notice Finalizes the migration by transitioning the state of the pool
180  function finalizeMigration() external override onlyOwner {
181      if (state == PoolState.Migrating) {
182          state = PoolState.TaxEnforcedAntiFarmer;
183          taxExpirationTime = block.timestamp + TAX_DURATION;
184          antiFarmerExpirationTime = block.timestamp + ANTI_FARMER_DURATION;
185      }
186  }

```

Listing 2.3: src/FlapTaxToken.sol

```

291 function _transfer(address from, address to, uint256 amount) internal override {
292     _liquidateTax(to);
293
294     if (state == PoolState.BondingCurve) {
295         require(!pools[from] && !pools[to], "Transfers to/from pools are restricted in
           BondingCurve state");
296         _plainTransfer(from, to, amount);
297     } else if (state == PoolState.Migrating) {
298         _plainTransfer(from, to, amount);
299     } else if (state == PoolState.TaxEnforcedAntiFarmer || state == PoolState.TaxEnforced) {
300         uint256 tax = _getTax(from, to, amount);
301         if (tax > 0) {
302             _taxedTransfer(from, to, amount, tax);
303         } else {
304             _plainTransfer(from, to, amount);
305         }
306     } else {
307         // TaxFree state
308         _plainTransfer(from, to, amount);
309     }
310 }

```

Listing 2.4: src/FlapTaxToken.sol

Impact Potential tax circumvention during the migration.

Suggestion Ensure that the pool migration including the liquidity addition is completed in a single transaction.

Feedback from the project The project stated that they would ensure the pool migration is finished in a single transaction.

2.1.3 Potential DoS due to the improper configuration in the contract `taxSplitter`

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `FlapTaxToken`, the function `_transfer()` invokes the function `_liquidateTax()` to perform tax liquidation and trigger the tax split implemented in the contract `taxSplitter`'s `fallback` function. In the tax split process, the tokens are split and sent to the pre-configured fee receivers and beneficiaries in the contract `taxSplitter`. However, this design is improper due to the improper configuration in the contract `taxSplitter`. Specifically, if the fee receivers and tax beneficiaries (i.e., pre-configured in the contract `taxSplitter`) implement a malicious or improper fallback function, the tax split process may fail. As a result, the improper configuration (e.g., tax beneficiary) in the contract `taxSplitter` may lead to potential DoS issues for the tax token transfers.

```
351         if (QUOTE_TOKEN == WETH) {
352             address[] memory path = new address[](2);
353             path[0] = address(this);
354             path[1] = WETH;
355
356             // Get expected output amount using getAmountsOut
357             uint256[] memory amounts = IUniswapRouter02(PCS_V2_ROUTER).getAmountsOut(
358                 taxAmount, path);
359             outputAmount = amounts[1];
360
361             try IUniswapRouter02(PCS_V2_ROUTER).
362                 swapExactTokensForETHSupportingFeeOnTransferTokens(
363                     taxAmount, outputAmount, path, taxSplitter, block.timestamp
364                 ) {
365                 _adjustLiquidationThreshold(taxAmount, outputAmount);
366             } catch (bytes memory reason) {
367                 emit TaxLiquidationError(reason);
368                 _plainTransfer(address(this), taxSplitter, taxAmount);
369             }
370         } else {
371             // quote token is ERC20
372             address[] memory path = new address[](2);
373             path[0] = address(this);
374             path[1] = QUOTE_TOKEN;
375
376             // Get expected output amount using getAmountsOut
377             uint256[] memory amounts = IUniswapRouter02(PCS_V2_ROUTER).getAmountsOut(
378                 taxAmount, path);
379             outputAmount = amounts[1];
380
381             try IUniswapRouter02(PCS_V2_ROUTER).
382                 swapExactTokensForTokensSupportingFeeOnTransferTokens(
383                     taxAmount, outputAmount, path, taxSplitter, block.timestamp
384                 ) {
385                 _adjustLiquidationThreshold(taxAmount, outputAmount);
386             }
387
388             {
389                 if (ERC20PermitUpgradeable(WETH).balanceOf(address(this)) > 0) {
390                     // We intentionally transfer 1 wei WBNB to the smart router
391                     // to allow it to unwrap weth and call the fallback function of the
392                     // splitter to trigger the split
393                     ERC20PermitUpgradeable(WETH).transfer(PCS_SMART_ROUTER, 1);
394                 }
395
396                 // then call smart router's unwrapWETH9 function
397                 ISmartRouter(PCS_SMART_ROUTER).unwrapWETH9(0, taxSplitter); // unwrap
398                                     WETH and send to taxSplitter
399             }
400         }
401     }
```

Listing 2.5: src/FlapTaxToken.sol

Impact The improper configuration (e.g., tax beneficiary) in the contract `taxSplitter` may

lead to potential DoS issues for the tax token transfers.

Suggestion Revise the tax liquidation logic in the contract `FlapTaxToken` to prevent potential DoS issues.

2.2 Recommendation

2.2.1 Correct the importing path of the contract `PoolAddress`

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `FlapTaxToken`, the import of the contract `PoolAddress` uses an incorrect path. It is recommended to correct the importing path of the contract `PoolAddress`.

```
8import {PoolAddress} from "../libraries/PoolAddress.sol";
```

Listing 2.6: `src/FlapTaxToken.sol`

Suggestion Correct the importing path of the contract `PoolAddress`.

2.2.2 Remove redundant code

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `FlapTaxToken`, the check (Line 329) of the pool state (i.e., the variable `state`) is redundant. It is recommended to remove it for better code readability.

```
329         if (block.timestamp > taxExpirationTime && state != PoolState.TaxFree) {
```

Listing 2.7: `src/FlapTaxToken.sol`

Suggestion Remove the redundant code.

2.2.3 Emit events for the state transition

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `FlapTaxToken`, the variable `state` is updated when time is reached. It is recommended to emit events for the state transition.

```
329         if (block.timestamp > taxExpirationTime && state != PoolState.TaxFree) {
330             state = PoolState.TaxFree;
331             taxRate = 0; // reset tax rate
332         } else if (block.timestamp > antiFarmerExpirationTime && state != PoolState.TaxEnforced
333                 ) {
334             state = PoolState.TaxEnforced;
335         }
```

Listing 2.8: `src/FlapTaxToken.sol`

```
180 function finalizeMigration() external override onlyOwner {
181     if (state == PoolState.Migrating) {
182         state = PoolState.TaxEnforcedAntiFarmer;
```

Listing 2.9: src/FlapTaxToken.sol

```
173 function startMigration() external override onlyOwner {
174     if (state == PoolState.BondingCurve) {
175         state = PoolState.Migrating;
```

Listing 2.10: src/FlapTaxToken.sol

Suggestion Emit events for the state transition.

2.2.4 Add validity checks in the function initialize()

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the `constructor()` and the function `initialize()`, the inputs (e.g., `params.PCS_V2_FACTORY`, `params.tax`, and `params.quoteToken`) are not checked to ensure they are not zero. It is recommended to add non-zero checks to prevent potential misoperations. Additionally, the variable `taxRate` is set based on the input `params.tax` in the function `initialize()`. It is recommended to add an upper-bound (i.e., `10000`) check for the variable `taxRate`. Furthermore, the input `params.taxDuration`, which is assigned to the variable `TAX_DURATION` in the function `initialize()`, should be checked to ensure that it is no less than the variable `ANTI_FARMER_DURATION`.

```
190 function initialize(InitParams memory params) external initializer {
191     __ERC20_init(params.name, params.symbol);
192     __ERC20Permit_init(params.name);
193     __Ownable_init();
194
195     metaURI = params.meta;
196     taxRate = params.tax;
197     taxSplitter = params.taxSplitter;
198     liquidationThreshold = START_LIQ_THRESHOLD;
199     notLiquidating = true;
200     TAX_DURATION = params.taxDuration;
```

Listing 2.11: src/FlapTaxToken.sol

```
258 function _getTax(address from, address to, uint256 amount) internal view returns (uint256) {
259     if (notLiquidating) {
260         if (state == PoolState.TaxEnforcedAntiFarmer) {
261             // Apply tax if transfer involves any pool
262             if (pools[from] || pools[to]) {
263                 return (amount * taxRate) / 10000;
264             }
265         } else if (state == PoolState.TaxEnforced) {
266             // Apply tax only if transfer involves v2Pool
```

```
267         if (from == mainPool || to == mainPool) {
268             return (amount * taxRate) / 10000;
269         }
270     }
271 }
272 return 0;
273 }
```

Listing 2.12: src/FlapTaxToken.sol

```
314 function _liquidateTax(address to) internal {
315     // Note: from == mainPool does not work and leads to a DOS vector.
316     // Uniswap v2 pair has a lock, if the user swaps WETH for the token, the pair first locks
317     // the pair and then
318     // transfers the token to the user (i.e from == mainPool). We could not liquidate in that "
319     // transfer" as the pair is already locked.
320     // Besides, the flash swap does not work even when swapping the token for WETH due to the
321     // same reason.
322     // For the tax token, you should follow the normal flow:
323     // (1) transfer token/WETH to the pair
324     // (2) call the pair's swap function.
325
326     if (
327         (state == PoolState.TaxEnforced || state == PoolState.TaxEnforcedAntiFarmer) // only
328         when tax is enforced
329         && notLiquidating // not in the middle of liquidation
330         && (to == mainPool) // possibly selling to the main v2 pool
331     ) {
332         // State transition
333         if (block.timestamp > taxExpirationTime && state != PoolState.TaxFree) {
334             state = PoolState.TaxFree;
335             taxRate = 0; // reset tax rate
336         } else if (block.timestamp > antiFarmerExpirationTime && state != PoolState.TaxEnforced) {
337             state = PoolState.TaxEnforced;
338         }
339     }
340 }
```

Listing 2.13: src/FlapTaxToken.sol

Suggestion Add validity checks in the function `initialize()`.

Feedback from the project The project stated that the inputs of the function `initialize()` are checked (i.e., non-zero and the upper-bound checks) in the caller contract.

2.3 Note

2.3.1 Potential centralization risks

Introduced by [Version 1](#)

Description In this project, several privileged roles (e.g., the role `Owner`) can conduct sensitive operations, which introduces potential centralization risks. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.3.2 Proxy deployment and implementation binding should be atomic

Introduced by [Version 1](#)

Description To prevent potential front-running attacks, it is recommended that proxy deployment and implementation binding be executed atomically within a single transaction. If these operations are performed separately, malicious actors could exploit the time window between deployment and binding to front-run the implementation binding transaction. An attacker could potentially bind their own malicious implementation to the newly deployed proxy contract, gaining unauthorized control over the protocol's upgrade mechanism and user funds. This race condition poses significant security risks including complete protocol compromise, fund theft, and unauthorized access to privileged functions.

2.3.3 Potential tax circumvention risks

Introduced by [Version 1](#)

Description In the contract [FlapTaxToken](#), the function `_getTax()` only returns a non-zero tax amount (i.e., the variable `tax`) for pools configured in the list `pools`. The project must be aware that the token transfers involving non-configured pools from different exchange protocols can circumvent the tax mechanism.

```
258 function _getTax(address from, address to, uint256 amount) internal view returns (uint256) {
259     if (notLiquidating) {
260         if (state == PoolState.TaxEnforcedAntiFarmer) {
261             // Apply tax if transfer involves any pool
262             if (pools[from] || pools[to]) {
263                 return (amount * taxRate) / 10000;
264             }
265         } else if (state == PoolState.TaxEnforced) {
266             // Apply tax only if transfer involves v2Pool
267             if (from == mainPool || to == mainPool) {
268                 return (amount * taxRate) / 10000;
269             }
270         }
271     }
272     return 0;
273 }
```

Listing 2.14: `src/FlapTaxToken.sol`

2.3.4 AMM compatibility with the fee-on-transfer tokens

Introduced by [Version 1](#)

Description In the contract [FlapTaxToken](#), when the pool state (i.e., the variable `state`) is set to `PoolState.TaxEnforcedAntiFarmer`, the tax mechanism is applied to all swaps in configured pools (i.e., the list `pools`). However, the project must be aware that not all AMM routers support fee-on-transfer tokens. For instance, while Uniswap V3 and V4 pools can technically accommodate fee-on-transfer tokens, their official routers may encounter failures for the swaps of the fee-on-transfer tokens (i.e., selling the [FlapTaxToken](#) via the router contract).

2.3.5 Proper operations during the migration process

Introduced by [Version 1](#)

Description In the contract [FlapTaxToken](#), when the pool state (i.e., the variable [state](#)) is switched to [PoolState.Migrating](#), the project is required to perform some operations (e.g., add liquidity to corresponding pools) before finalizing the migration. The project must ensure the specific operations are properly performed with reasonable slippage check and appropriate liquidity amount before finalizing the migration.

